PPT with extra info



PHP Variables(pinaresearch ni sir...extra info lng)

What is it?

> A variable in PHP is a named storage location used to hold a value that a program can work with.
> The value may be a string, integer, floating-point number, Boolean, array, object, or another supported PHP type.
> PHP represents variables using a dollar sign ($) followed by the variable name, and variable names are case-sensitive.

Example:

```
$name = "Reeyana";
$age = 18;
//$name contains the string "Reeyana" and $age contains the integer 18.
//The variable name gives PHP a way to refer to the stored value later in the program.
```

How do PHP variables work?

> PHP variables generally work through three basic actions: creating a variable, assigning a value, and using or changing that value.

Step 1: A variable is created

> You create a variable by writing $ followed by a valid variable name.
> PHP variable names must begin with a letter or underscore and can then contain letters, numbers, and underscores.
> They are also case-sensitive, meaning $name and $Name are considered different variables.

Step 2: A value is assigned

> The assignment operator = is used to give a variable a value.

- You can also assign numbers.

Step 3: PHP determines the variable's type

> One important feature of PHP is that it is dynamically typed.

- This means that you normally don't have to specify the variable's data type when creating it.
- PHP determines the type based on the value assigned to the variable at runtime.

> PHP's documentation explains that the type of a variable is determined by the value it currently stores.

Step 4: The variable can be used

> Once a value is stored, PHP can use the variable in expressions, calculations, comparisons, output, and other operations.

- The variables allow PHP to calculate the result without having to repeatedly write the actual numbers.

Step 5: The value can be changed

> A variable is not permanently stuck with its original value.

Why does PHP use $ for variables?

> The $ symbol tells PHP that the identifier that follows is a variable.
> The PHP Manual specifically defines variables as being represented by a dollar sign followed by the variable's name.

Does PHP require you to declare the data type?

> No, not normally.

- The official PHP documentation describes PHP as a dynamically typed language, meaning the type is normally determined at runtime. However, modern PHP also supports type declarations in appropriate situations, particularly for function arguments, return values, properties, and other language features.

Does PHP have data types?

> Yes. PHP has data types, but you normally do not have to declare the type when creating a variable because PHP determines it from the value.

What types of values can PHP variables contain?

> **Boolean** – true or false
> **Integer** – whole numbers
> **Float** – numbers with decimal values
> **String** – text
> **Array** – collections of values
> **Object** – instances of classes
> **NULL** – represents no value
> **Resource** – a special type used for > certain external resources
> **Callable** – something that can be called as a function

Type Juggling

> PHP can sometimes automatically convert a value from one type to another when an operation requires it. This behavior is called type juggling. The PHP documentation explains that PHP may attempt to convert values into a type appropriate for the context in which they are being used.

Variable Scope

> Refers to where a variable can be accessed in a PHP program.
> PHP has global and function/local scopes, among other scope-related behavior.
> A variable created **inside** a function is normally available **only within** that function.
> A variable created **outside** functions belongs to the **global scope**.

- This matters because a variable does not necessarily exist or work everywhere in a program.

Why are variables important in PHP?

> Variables are important because websites and programs frequently need to work with changing information.
> Without variables, programmers would have to repeatedly work directly with values rather than giving those values meaningful names.

> Variables therefore make programs more flexible, readable, and capable of processing changing data.

What happens when one variable is assigned to another?

> The PHP Manual explains that assignment by value means the value of the original expression is assigned to the destination variable.
> PHP also supports assignment by reference, where a variable can refer to another variable using &.

Output Variables

echo

> It has no return value, while print has a return value of 1 so it can be used in expressions
> It can take multiple parameters, while print can take only one argument
> It is marginally faster than print
> The echo function can be used with or without parentheses: echo or echo().

print

> The print function can be used with or without parentheses: print or print().



Introduction to PHP

Humburger analogy

> Stacking of a website
- HTML
- CSS
- PHP
- JavaScript

PHP facts

> Stands for: Hypertext Preprocessor
> It is a server side scripting language designed for websites.
> Originally made for Personal Home Page
> Created on 1994 by Rasmus Lerdorf

Why choose PHP?

> PHP **runs** on different platforms
- Windows
- Linux
- Unix

> PHP is **compatible** with almost all servers used today
- Apache
- IIS

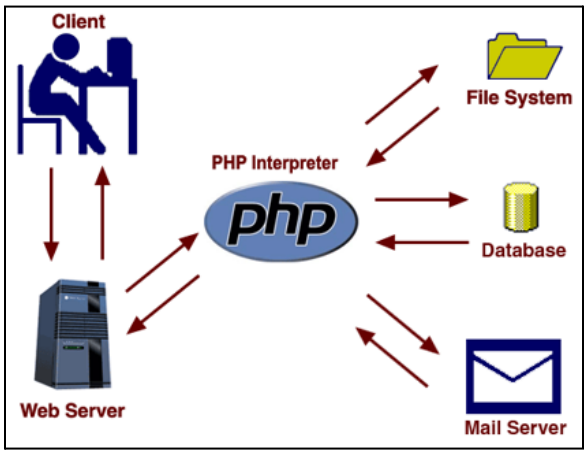
> PHP **supports** many databases
- MySQL
- Informix
- Oracle
- Sybase
- Solid
- PostgreSQL
- Generic ODBC

> PHP is an **open source** software
> PHP is **free** to download and use
> PHP is **easy to learn and runs efficiently** on the server side

PHP File

> Contains text, HTML tags, and scripts
> They are returned to the browser as plain HTML
> file extension = ".php"

PHP works by



1. Install Apache (or IIS), PHP, and MySQL on your own server
   - Apache, PHP and MySql can be installed easily by using the installation package rather than installing them individually.
   - **XAMPP and XAMPPLITE** are examples of PHP/MySQL packages. They are available for free download.
2. Test it
   - Create a PHP document using Adobe Dreamweaver

or Notepad with this content.

```
<?php
        Phpinfo();
?>
```

3. Create a folder inside the htdocs found in XAMPP/XAMPPLITE's Folder and save as your php page as test.php

Tools to Build PHP Scripts (ppt base)

> Programming Editor
> Integrated Development Environment (IDE)

Tools to Build PHP Scripts (sir tomas base)

> Text Editor
> XAMPP
- X = cross platform
- A = Apache
- M = Maria DB
- P = PHP
- P = Perl

> Browser

PHP Tags

> To open a block of PHP code in a page you can use one of these four sets of opening and closing tags:

Tags	Syntax
Short Style	<? [code here] ?>
XML Style	<?php [code here] ?>
Script Style	<script language="php"> [code here] </script>
ASP Style	<% [code here] %>

PHP Statements and Whitespaces

> an instruction that tells PHP to perform an action
> The ***PHP interpreter*** condenses all sequential whitespace in PHP scripts to a single whitespace.
- Code between <?php and ?> is processed by the PHP interpreter.

Code Block

> A series of statements enclosed between two braces

Echo command

> used to print text back to the browser.
> double quotes for printing

Simple code

```
<?php
        echo "Hello, World!";
?>
```

Simple code with variable

```
<?php
        $name = “Reeyana”;
        echo "My name is $name.";
?>
```

Simple code with variable and computation

```
<?php
        $num1 = 2;
        $num2 = 5;
        $sum = $num1 + $num2;
        echo "The sum is $sum.";
?>
```



PHP Comments, Constants, and Variables

Comments

> Use to describe scripts
> Two forms:
- **Single-line comments** begin with a double slash (//).
- **Multi-line comments** begin with "/*" and end with "*/"

Variables

> used to store information like text strings, numbers or arrays
> all the variables begin with a dollar sign "$" and the value can be assigned using the "=" operator
> SYNTAX:

$var_name = “Value”;

> Variables can, but do not need, to be declared before assignment

Variable Types

Variable Type	Explanation
Integer	Whole number
Double	Real number
String	String of characters
Boolean	True or false
Array	List of items
Object	Instance of a class

Rules for Variable Names

> must start with a letter or an underscore "_".
> can only contain alpha-numeric characters and underscores (a-z, A-Z, 0-9, and _).
> should not contain spaces
- If a variable name is more than one word, it should be separated with an underscore ($my_string), or with capitalization ($myString).

> Variables are case sensitive.

Variable Variables

> a variable whose name is contained in another variable:

```
$name = ‘foo’;
$$name = ‘bar’;
        echo $foo; //displays bar
```

```
$x = “Eric”;
$$x = “Seufert”; //$Eric = “Seufert”;
echo “$x ${$x}”;
//would produce the same output as, echo “$x $Eric”;
//which is: Eric Seufert
```

> The curly braces around the variable variable is the standard syntax for outputting variable variables.
> Omitting the curly braces would have resulted with the following output:

Eric $Eric

Variable Type Strength

> PHP is weakly typed:
- variables do not need to be assigned a type at the time they are declared.

> PHP variable is determined by the value the variable holds and the way in which it is used.

Variable Scope

> A variable's scope determines the locations from which the variable can be accessed.

Scope	Explanation
superglobal	Superglobal variables are predefined arrays, including $_POST and $_GET. They are accessible from anywhere on the page.
global	Global variables are visible throughout the script in which they are declared. However, they are not visible within the functions in the script unless they are re-declared within the function as a global variables.
function	Variables in the function scope are called local variables. Local variables are local to the function in which they are declared.

Super Globals

Super Globals

$_GET	Variables passed into a page on the query string.
$_POST	Variables passed in to a page through a form using the post method.
$_COOKIE	Cookie variables.
$_FILES	Variables containing information about the uploaded file.
$_ENV	PHP Environment variable that returns the name of the host server.
$_REQUEST	Variable passed into page through forms using string and cookies.
$_SERVER	Server environment variable that returns the URL of the referring page.
$_SESSION	Session variables.

1. `$_GET`
   - **Definition:** An associative array of variables passed to the current script via the HTTP query string (the URL).
   - **Visibility:** Data is completely visible in the browser's address bar (e.g., `page.php?id=5&user=john`).
   - **Best Used For:** Non-sensitive operations like page filtering, search queries, pagination, or sorting.
   - **Limitations:** Should never be used for passwords or sensitive data. It has a strict character limit (typically around 2,000 characters depending on the browser/server).
   - **Code Example:**

```php
// URL: index.php?name=Alex
echo "Hello " . htmlspecialchars($_GET['name']);
```

Use code with caution.

2. `$_POST`
   - **Definition:** An associative array of variables passed to the current script via the HTTP POST method.
   - **Visibility:** Data is invisible in the URL; it is sent within the HTTP request body.
   - **Best Used For:** Submitting forms, inserting/updating database records, uploading files, and sending sensitive information (like passwords).
   - **Advantages:** No data length restrictions. It is much more secure than `$_GET`, though it still requires server-side validation.
   - **Code Example:**

```php
// Form submission
$username = $_POST['username'];
```

Use code with caution.

3. `$_COOKIE`
   - **Definition:** An associative array of variables passed to the current script via HTTP Cookies.
   - **Mechanism:** Data is stored in the **user's browser** by the server and sent back to the server with every subsequent request.
   - **Best Used For:** Storing user preferences, tracking identifiers, or implementing a "Remember Me" login feature.
   - **Security Note:** Cookies can be easily edited or spoofed by the user. Never store raw passwords or sensitive permissions inside a cookie.
   - **Code Example:**

```php
// Checking a pre-set cookie named "user"
if(isset($_COOKIE['user'])) {
    echo "Welcome back, " . $_COOKIE['user'];
}
```

4. `$_FILES`
   - **Definition:** An associative array containing items uploaded to the current script via the HTTP POST method.
   - **Requirement:** The HTML form must contain the attribute `enctype="multipart/form-data"` and use `method="POST"`.
   - **Array Structure:** For an input named `file1`, `$_FILES['file1']` contains a nested array with 5 key pieces of data:
     - `$_FILES['file1']['name']` – Original file name.
     - `$_FILES['file1']['type']` – Mime type (e.g., `image/jpeg`).
     - `$_FILES['file1']['tmp_name']` – The temporary path where the file is stored on the server.
     - `$_FILES['file1']['error']` – The error code (0 means success).
     - `$_FILES['file1']['size']` – The file size in bytes.

5. `$_ENV`
   - **Definition:** An associative array of variables passed to the current script from the environment under which the PHP parser is running.
   - **Best Used For:** Storing system-specific configurations, external API keys, database credentials, or framework settings (often loaded from a `.env` file).
   - **Troubleshooting Tip:** If `$_ENV` returns empty, verify that the `variables_order` setting in your `php.ini` file includes the letter **"E"** (e.g., `variables_order = "EGPCS"`).
   - **Code Example:**

```php
$db_password = $_ENV['DB_PASSWORD'];
```

Use code with caution.

6. `$_REQUEST`

- **Definition:** A default associative array that contains the combined contents of `$_GET`, `$_POST`, and `$_COOKIE`.
- **Use Case:** Used when a script handles data but does not care whether it arrived via a URL link (`GET`) or a form submission (`POST`).
- **Security Note:** It is generally considered best practice to use the specific `$_GET` or `$_POST` array rather than `$_REQUEST` to prevent unintended data overrides and keep code intent clear.

7. `$_SERVER`

- **Definition:** An associative array containing information such as headers, paths, script locations, and server configurations.
- **Key Indices to Memorize:**
  - `$_SERVER['REQUEST_METHOD']` – Returns the request type (e.g., `GET`, `POST`).
  - `$_SERVER['HTTP_HOST']` – Returns the Host header from the current request.
  - `$_SERVER['HTTP_REFERER']` – The URL of the page that referred the user to the current page.
  - `$_SERVER['REMOTE_ADDR']` – The IP address of the user viewing the current page.
  - `$_SERVER['PHP_SELF']` – The filename of the currently executing script relative to the document root.

8. `$_SESSION`

- **Definition:** An associative array containing session variables available to the current script.
- **Mechanism:** Unlike cookies, session data is stored **securely on the server**. A unique session ID is generated and stored in a user cookie to link the browser to the server data.
- **Requirement:** You must call `session_start();` at the very top of your script before accessing or setting any `$_SESSION` data.
- **Best Used For:** Keeping users logged in across multiple pages, managing shopping carts, and carrying user state safely.
- **Code Example:**

```php
session_start();
$_SESSION['user_id'] = 42; // Set session data
```

PHP Keywords

and	echo	list
as	else	new
break	empty	or
case	eval	print
class	exit	require
const	for	return
continue	foreach	switch
declare	function	use
default	global	var
die	if	while
do	include	

Constants

> They are like variables except that, once assigned a value, they cannot be changed.

> They are created using the define() function and by convention (but not by rule) are in all uppercase letters.
> Can be accessed from anywhere on the page.
> SYNTAX:

define('CONST_NAME',VALUE);

> They are can also be created using const
> SYNTAX:

const CONST_NAME = VALUE;

> both ways can be accessed anywhere

- Yet they are evaluated in different times
  - const compiles before the code runs (Compile-time)
  - define() compiles while the code is running (Runtime)
- Yet const cannot be used in conditional blocks while define() can be used



PHP Operators

Arithmetic

Operator	Name	Example
+	Addition	$a + $b;
-	Subtraction	$a - $b;
*	Multiplication	$a * $b;
/	Division	$a / $b;
%	Modulus / Remainder	$a % $b;

String

Operator	Name	Example
.	Concatenation	$a . $b;

Assignment

Operator	Name	Example
=	Assignment	$a = 1;
+= -= *= /=	Combination Assignment	$a += 1; $a -= 1; $a *= 1; $a /= 1;
++	Increment by 1	$a ++;
--	Decrement by 1	$a --;

Comparison

Operator	Name	Example
==	is equal	$a == $b
!=	is not equal	$a != $b
<	is less than	$a < $b
>	is greater than	$a > $b
<=	is less than or equal to	$a <= $b
>=	is greater than or equal to	$a >= $b
===	identical	$a === $b
!==	not identical	$a !== $b

Logical

Operator	Name	Example
!	NOT	!$a
&&	AND	$a && $b
||	OR	$a || $b

Single Quotes vs. Double Quotes

> ***Double quotation marks*** will replace the variable name with its value and will escape a character (\$$total)
> ***Single quotes*** will always display exactly what is typed

- except for the escaped single quote (\') and escaped backslash (\\)



PHP Control Structures

Conditional Processing

> Allows programmers to output different code based on specific conditions.

Type of conditionals

if statement

> to execute some code only if a specified condition is true.
> SYNTAX:

```
if (condition)
        code to be executed if condition is
        true;
```

> EXAMPLE:

```
if ($x == 1)
        echo "True";
```

if...else statement

> to execute some code if a condition is true and another code if the condition is false
> SYNTAX:

```
if (condition)
        code to be executed if condition is
        true;
else
        code to be executed if condition is
        false;
```

> EXAMPLE:

```
if($x == 1)
        echo "True";
else
```

```
        echo “False”;
```

Short form of if – else statement using ternary operator

> SYNTAX:

```
(condition ? Value_if_true :
Value_if_false)
```

> EXAMPLE:

```
<?php
        $n = 15;
        $a = ($n % 2 ==0 ? ‘Even
                Number’: ‘Odd Number’)
?>
```

if…elseif….else statement

> to select one of several blocks of code to be executed

> SYNTAX:

```
if (condition)
        code to be executed if condition is
        true;
elseif (condition)
        code to be executed if condition is
        true;
else
        code to be executed if condition is
        false;
```

> EXAMPLE:

```
if($x == 1)
        echo “One”;
elseif($x == 2)
        echo “Two”;
else
        echo “Three”;
```

switch statement

> to select one of several blocks of code to be executed

> SYNTAX:

```
switch (n) {
    case label1:
        code to be executed if n=label1;
        break;
    case label2:
        code to be executed if n=label2;
        break;
    default:
        code to be executed if n is
        different from both label1 and
        label2;
}
```

> EXAMPLE:

```
switch($x)
{
        case 1:
                echo “One”;
                break;
        case 2:
                echo “Two”;
                break;
        default:
                echo “Three”;
                break;
}
```

Loops

> They execute a block of code a specified number of times, or while a specified condition is true.

Type of loops

while loop

> Executes a block of code while a condition is true.

> SYNTAX:

```
while (condition)
{
        code to be executed;
}
```

> EXAMPLE:

```
$x = 1;
while ($x < 5)
```

```
{
        $x = $x + 1;
}
echo $x;
```

do...while loops

> Always execute the block of code once, it will then check the condition, and repeat the loop while the condition is true.

> SYNTAX:

```
do
{
       code to be executed;
}
       while (condition);
```

> EXAMPLE:

```
$x = 1;
do
{
     $x = $x + 1;
}
     while ($x < 5)
       echo $x;
```

for loops

> used when you know in advance how many times the script should run. .

> SYNTAX:

```
for (init; condition; increment)
{
        code to be executed;
}
```

> EXAMPLE:

```
for($x = 1; $x < 5; $x++)
{
        echo $x . "<br />" ;
}
```

Break and Continue

> **break:** breaks completely out of a loop and continues with the script statements after the loop

> SYNTAX/EXAMPLE:

```
    for ($a=1; $a < 6; $a++)
{
        echo $a;
    if ($a > 3)
{
    break;
  }
}
```

> **continue:** skips to the end of the loop where the condition is tested. If the condition tests positive, the script continues from the top of the loop

> SYNTAX/EXAMPLE:

```
    for ($a=1; $a < 6; $a++)
{
    if ($a == 3)
{
   continue;
 }
   echo $a;
}
```

CODES



Frontend and Backend codes

Notes

> Backend = PHP Logic
> Frontend = HTML, CSS, a bit of javascript
> always use .php

Login page scenario

FRONTEND

```php
<html>
    <head>
        <title>Login Module</title>
    </head>
    <body>
        <form action="login.php" method="post">
            <table>
                <tr>
                    <td>Enter Username:</td>
                    <td><input type="text" name="username" required></td>
                </tr>
                <tr>
                    <td>Enter Password:</td>
                    <td><input type="password" name="password" required></td>
                </tr>
                <tr>
                    <td><input type="submit" value="Login"></td>
                    <td><input type="reset" value="Clear"></td>
                </tr>
            </table>
        </form>
    </body>
</html>
```

> file name: home.php

BACKTEND

```php
<?php
    $username = $_POST['username'];
    $password = $_POST['password'];
    $dafaultUsername = "admin";
    $defaultPassword = "admin123";
    if($username == $dafaultUsername && $password == $defaultPassword){
?>
        <script>
            alert("Login Successful");
            window.location.href = "index.php";
        </script>
<?php
    }
    else{
?>
        <script>
            alert("Login Failed");
            window.location.href = "home.php";
        </script>
<?php
    }
?>
```

> file name: login.php

Status highlighting of record scenario

FRONTEND AND BACKEND

```php
<html>
    <head>
        <title>Record Management</title>
    </head>
    <body>
        <table>
            <thead>
                <tr>
                    <th>Course</th>
                    <th>Section</th>
                    <th>Status</th>
                </tr>
            </thead>
            <tbody>
                <tr
                    <?php
                        $stat = "1";
                        if($stat == 1){
                            echo "style='background: green;'";
                        }
                        else{
                            echo "style='background: red;'";
                        }
                    ?>
                >
                    <td>BSIT</td>
                    <td>1A</td>
                    <td>Active</td>
                </tr>
                <tr
                    <?php
                        $stat = "0";
                        if($stat == 1){
                            echo "style='background: green;'";
                        }
                        else{
                            echo "style='background: red;'";
                        }
                    ?>
                >
                    <td>BSCS</td>
                    <td>2B</td>
                    <td>Inactive</td>
                </tr>
                <tr
                    <?php
                        $stat = "1";
                        if($stat == 1){
                            echo "style='background: green;'";
                        }
                        else{
                            echo "style='background: red;'";
                        }
                    ?>
                >
                    <td>BSIS</td>
                    <td>3C</td>
                    <td>Active</td>
                </tr>
            </tbody>
        </table>
    </body>
</html>
```

> file name: record.php
> note: all codes between <?php ?> is backend

Registration module scenario (with javascript)

FRONTEND

```html
<html>
    <head>
        <title>Sample Registration</title>
        <link rel="stylesheet" href="style.css">
        <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
    </head>
    <body>
        <h2>Create Account</h2>
        <table>
            <tr>
                <td>Full Name:</td>
                <td><input type="text" name="fullname" id="fullname" placeholder="John Doe"></td>
            </tr>
            <tr>
                <td> </td>
                <td colspan=2><div id="fullname_error"></div></td>
            </tr>
            <tr>
                <td>Username:</td>
                <td><input type="text" name="username" id="username" placeholder="johndoe_123"></td>
            </tr>
            <tr>
                <td> </td>
                <td colspan=2><div id="username_error"></div></td>
            </tr>
            <tr>
                <td>Email:</td>
                <td><input type="email" name="email" id="email" placeholder="sample@email.com"></td>
            </tr>
            <tr>
                <td> </td>
                <td colspan=2><div id="email_error"></div></td>
            </tr>
            <tr>
                <td>Phone:</td>
                <td><input type="tel" name="phone" id="phone" placeholder="09123456789"></td>
            </tr>
            <tr>
                <td> </td>
                <td colspan=2><div id="phone_error"></div></td>
            </tr>
            <tr>
                <td>Password (Min. 8 chars, 1 letter & 1 number):</td>
                <td><input type="password" name="password" id="password" placeholder="Min. 8 chars, 1 letter & 1 number"></td>
            </tr>
            <tr>
                <td> </td>
                <td colspan=2><div id="password_error"></div></td>
            </tr>
            <tr>
                <td>Confirm Password:</td>
                <td><input type="password" name="confirm_password" id="confirm_password" placeholder="Re-type password"></td>
            </tr>
            <tr>
                <td> </td>
                <td colspan=2><div id="confirm_password_error"></div></td>
            </tr>
            <tr>
                <td colspan="3">
                    <button class="btn" id="reg_btn">Register Now</button>
                </td>
            </tr>
        </table>
        <script src="validation.js"></script>
    </body>
</html>
```

> file name: index.php

```javascript
$(document).ready(function() {

    $("#reg_btn").on("click", function(e) {
        let isValid = true;

        let fullname = $("#fullname").val();
        let username = $("#username").val();
        let email = $("#email").val();
        let phone = $("#phone").val();
        let password = $("#password").val();
        let confirm_password = $("#confirm_password").val();

        // Full Name
        if (fullname.length < 3) {
            $('#fullname_error').text("Full name must be at least 3 characters long.");
            isValid = false;
        } else if (!/^[a-zA-Z\s]+$/.test(fullname)) {
            $('#fullname_error').text("Full name can only contain letters and spaces.");
            isValid = false;
        }

        // Username
        if (!/^[a-zA-Z0-9_]{4,20}$/.test(username)) {
            $('#username_error').text("Username must be 4-20 characters long and can only contain letters, numbers, and underscores.");
            isValid = false;
        }

        // Email
        if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
            $('#email_error').text("Please enter a valid email address.");
            isValid = false;
        }

        // Phone
        if (!/^[0-9]{10,15}$/.test(phone)) {
            $('#phone_error').text("Phone number must be between 10 to 15 digits.");
            isValid = false;
        }

        // Password
        if (password.length < 8) {
            $('#password_error').text("Password must be at least 8 characters long.");
            isValid = false;
        } else if (!/(?=.*[A-Za-z])(?=.*\d)/.test(password)) {
            $('#password_error').text("Password must contain at least one letter and one number.");
            isValid = false;
        }

        // Confirm Password
        if (confirm_password !== password) {
            $('#confirm_password_error').text("Passwords do not match.");
            isValid = false;
        }

        if (isValid) {

            let url = "process.php?fullname=" + fullname +
            "&username=" + username +
            "&email=" + email +
            "&phone=" + phone +
            "&password=" + password +
            "&confirm_password=" + confirm_password;
            window.location.href = url;

        }
    });
});
```

> file name: validation.js

BACKEND

```php
<?php
    $fullname = $_GET['fullname'];
    $username = $_GET['username'];
    $email = $_GET['email'];
    $phone = $_GET['phone'];
    $password = $_GET['password'];
    $confirm_password = $_GET['confirm_password'];

    echo "Full Name: $fullname<br>";
    echo "Username: $username<br>";
    echo "Email: $email<br>";
    echo "Phone: $phone<br>";
    echo "Password: $password<br>";
    //Database insertion logic here
?>

```

> file name: process.php

Student Grade Evaluation System scenario (if-elseif-else)

FRONTEND

```php
<!DOCTYPE html>
<html>
<head>
    <title>Student Grade Evaluation</title>
</head>
<body>

    <h2>Student Grade Evaluation System</h2>

    <form action="scenario1result.php" method="post">

        <label>Student Name:</label>
        <input type="text" name="student_name" required>
        <br><br>

        <label>Subject:</label>
        <input type="text" name="subject" required>
        <br><br>

        <label>Score:</label>
        <input type="number" name="score" min="-100" max="200" required>
        <br><br>

        <input type="submit" value="Evaluate Grade">

    </form>

</body>
</html>
```

> file name: scenario1index.php

BACKEND

```php
<?php

$student_name = $_POST['student_name'];
$subject = $_POST['subject'];
$score = $_POST['score'];

if ($score > 100 || $score < 0) {
    $grade = "N/A";
    $remarks = "Invalid Score";
}
elseif ($score >= 90) {
    $grade = "A";
    $remarks = "Excellent - With Honors";
}
elseif ($score >= 80) {
    $grade = "B";
    $remarks = "Very Good";
}
elseif ($score >= 70) {
    $grade = "C";
    $remarks = "Good";
}
elseif ($score >= 60) {
    $grade = "D";
    $remarks = "Passed - Needs Improvement";
}
else {
    $grade = "F";
    $remarks = "Failed";
}

?>
```

```php
<!DOCTYPE html>
<html>
<head>
    <title>Grade Result</title>
</head>
<body>

    <h2>Student Grade Result</h2>

    <p><strong>Student Name:</strong> <?php echo htmlspecialchars($student_name); ?></p>
    <p><strong>Subject:</strong> <?php echo htmlspecialchars($subject); ?></p>
    <p><strong>Score:</strong> <?php echo $score; ?></p>
    <p><strong>Grade:</strong> <?php echo $grade; ?></p>
    <p><strong>Remarks:</strong> <?php echo $remarks; ?></p>

    <?php
    if ($grade == "A" && $score == 100) {
        echo "<p><strong>Perfect Score!</strong></p>";
    }
    ?>

    <br>
    <a href="scenario1index.php">Evaluate Another Student</a>

</body>
</html>
```

> file name: scenario1result.php

Online Shop Discount Calculator scenario (Nested if)

FRONTEND

```html
<!DOCTYPE html>
<html>
<head>
    <title>Online Shop Discount Calculator</title>
</head>
<body>

    <h2>Online Shop Discount Calculator</h2>

    <form action="scenario2result.php" method="post">

        <label>Customer Name:</label>
        <input type="text" name="customer_name" required>
        <br><br>

        <label>Total Purchase:</label>
        <input type="number" name="purchase" min="0" step="0.01" required>
        <br><br>

        <label>Member:</label>
        <input type="radio" name="member" value="Yes" required> Yes
        <input type="radio" name="member" value="No"> No
        <br><br>

        <input type="submit" value="Calculate Discount">

    </form>

</body>
</html>
```

> file name: scenario2index.php

BACKEND

```php
<?php

$customer_name = $_POST['customer_name'];
$purchase = $_POST['purchase'];
$member = $_POST['member'];

if ($member == "Yes") {

    if ($purchase >= 5000) {
        $discount = 20;
    } else {
        $discount = 10;
    }

} else {

    if ($purchase >= 5000) {
        $discount = 5;
    } else {
        $discount = 0;
    }
}

$discount_amount = $purchase * ($discount / 100);
$final_amount = $purchase - $discount_amount;

?>
```

```php
<!DOCTYPE html>
<html>
<head>
    <title>Discount Result</title>
</head>
<body>

    <h2>Discount Calculation Result</h2>

    <p><strong>Customer Name:</strong>
        <?php echo htmlspecialchars($customer_name); ?>
    </p>
    <p><strong>Total Purchase:</strong>
        ₱<?php echo number_format($purchase, 2); ?>
    </p>
    <p><strong>Member:</strong>
        <?php echo $member; ?>
    </p>
    <p><strong>Discount:</strong>
        <?php echo $discount; ?>%
    </p>
    <p><strong>Discount Amount:</strong>
        ₱<?php echo number_format($discount_amount, 2); ?>
    </p>
    <p><strong>Final Amount:</strong>
        ₱<?php echo number_format($final_amount, 2); ?>
    </p>

    <br>
    <a href="scenario2index.php">Calculate Another Purchase</a>

</body>
</html>
```

> file name: scenario2result.php

Food Menu Price List scenario (switch case)

FRONTEND

```php
<!DOCTYPE html>
<html>
<head>
    <title>Food Menu Price List</title>
</head>
<body>

    <h2>Food Menu</h2>

    <form action="scenario3result.php" method="post">

        <label for="code">Select Menu Code:</label>
        <select name="code" id="code">
            <option value="A">A - Chicken Adobo + Rice</option>
            <option value="B">B - Pork Sinigang + Rice</option>
            <option value="C">C - Beef Steak + Rice</option>
            <option value="D">D - Vegetable Curry + Rice</option>
        </select>

        <br><br>

        <label for="qty">Quantity:</label>
        <input type="number" name="qty" id="qty" min="1" required>

        <br><br>

        <button type="submit">Calculate Total</button>

    </form>

</body>
</html>
```

> file name: scenario3index.php

BACKEND

```php
<?php
$code = $_POST['code'];
$qty = $_POST['qty'];

switch ($code) {
    case "A":
        $meal = "Chicken Adobo + Rice";
        $price = 75;
        break;

    case "B":
        $meal = "Pork Sinigang + Rice";
        $price = 85;
        break;

    case "C":
        $meal = "Beef Steak + Rice";
        $price = 95;
        break;

    case "D":
        $meal = "Vegetable Curry + Rice";
        $price = 60;
        break;

    default:
        echo "<h2>Invalid Menu Code.</h2>";
        exit;
}
if ($qty <= 0) {
    echo "<h2>Error: Quantity must be greater than 0.</h2>";
    exit;
}

$total = $price * $qty;
?>
```

```php
<!DOCTYPE html>
<html>
<head>
    <title>Order Result</title>
</head>
<body>

    <h2>Order Summary</h2>

    <p><strong>Menu Code:</strong> <?php echo $code; ?></p>
    <p><strong>Meal:</strong> <?php echo $meal; ?></p>
    <p><strong>Price:</strong> ₱<?php echo $price; ?></p>
    <p><strong>Quantity:</strong> <?php echo $qty; ?></p>
    <p><strong>Total:</strong> ₱<?php echo $total; ?></p>

    <br>

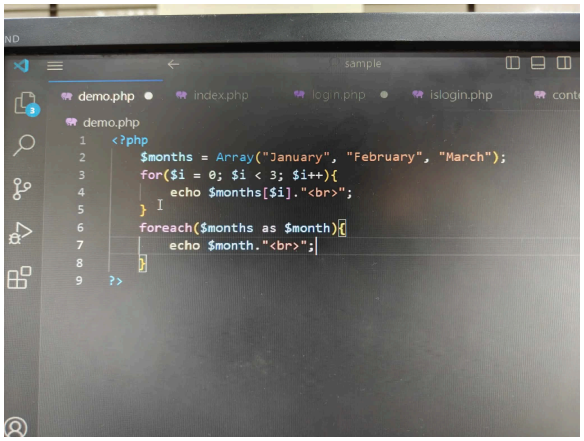
    <a href="scenario3index.php">Back to Menu</a>

</body>
</html>
```

> file name: scenario3result.php

USE OF ARRAY scenario

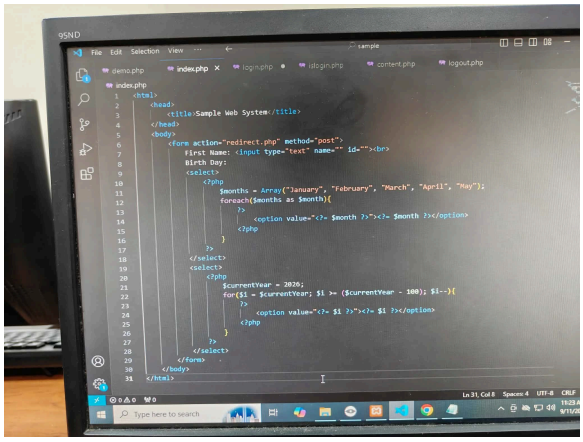
BACKEND



> file name: demo.php

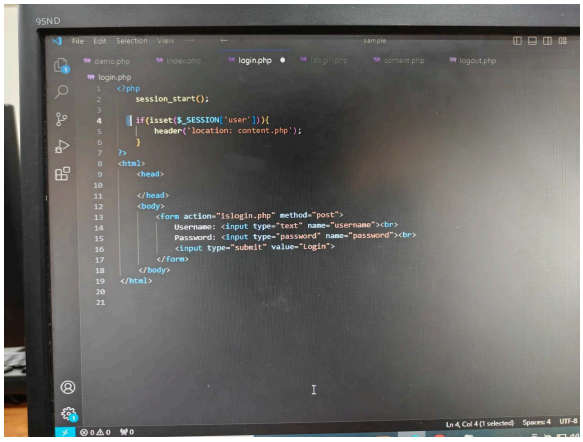
Sample web system scenario

FRONTEND

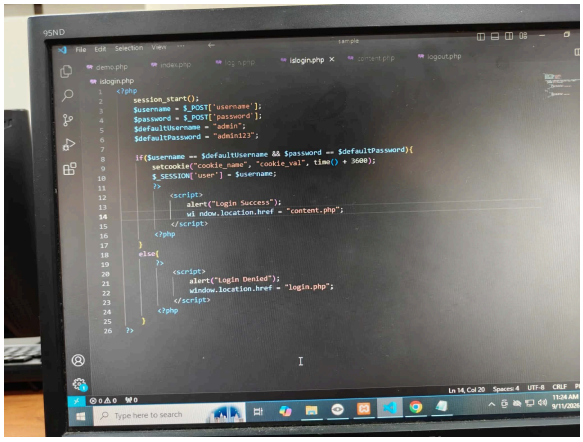


> file name: index.php

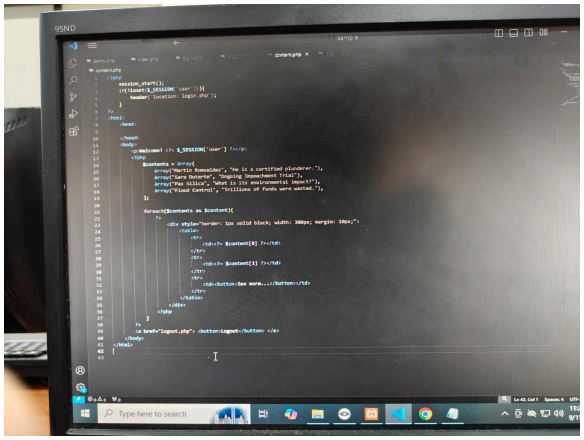
BACKEND



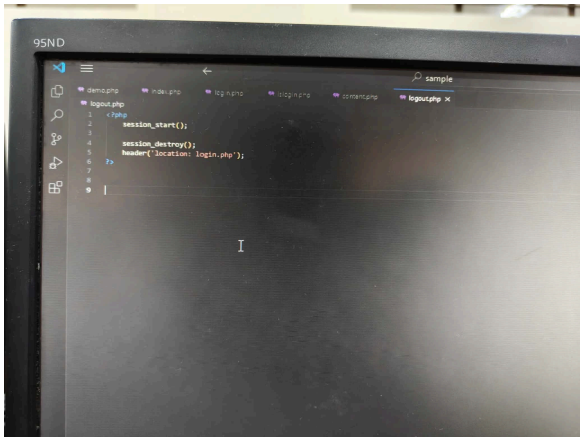
> file name: login.php



> file name: islogin.php



> file name: content.php



> file name: logout.php